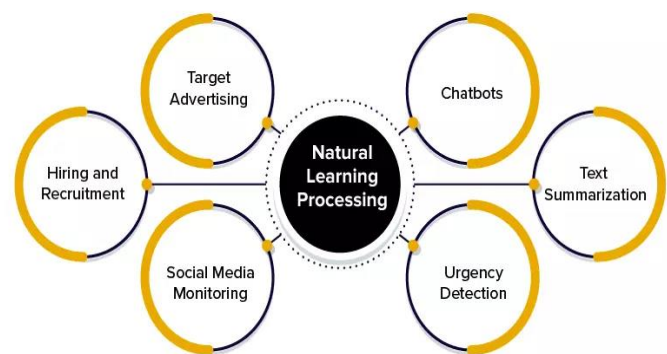


Natural Language processing

What is natural language processing?

- Natural language processing (NLP) refers to the branch of computer science—and more specifically, the branch of artificial intelligence or AI—concerned with giving computers the ability to understand text and spoken words in much the same way human beings can.
- NLP combines computational linguistics—rule-based modelling of human language—with statistical, machine learning, and deep learning models.
- Together, these technologies enable computers to process human language in the form of text or voice data and to ‘understand’ its full meaning, complete with the speaker or writer’s intent and sentiment.



NLP USE CASES Natural language processing is the driving force behind machine intelligence in many modern real-world applications. Here are a few examples:

- **Spam detection:** NLP's text classification capabilities can be used to scan emails for language that often indicates spam or phishing. These indicators can include overuse of financial terms, characteristic bad grammar, threatening language, inappropriate urgency, misspelled company names, and more.
- **Machine translation:** Machine translation involves translating words/sentences in one language to another. Effective translation has to capture accurately the meaning and tone of the input language and translate it to text with the same meaning and desired impact in the output language. Example: Google translate
- **Speech Recognition:** This is the process of mapping acoustic speech signals to a set of words. Difficulty arises due to wide variations in the pronunciations of words, homonym (e.g., dear and deer) and acoustic ambiguities (e.g., in the rest and interest)

- **Speech Synthesis:** It refers to automatic production of speech (uttering sentences in natural language). Such systems can read out your mail or messages for you.
- **Virtual agents and chatbots:** Virtual agents such as Apple's Siri and Amazon's Alexa use speech recognition to recognize patterns in voice commands and natural language generation to respond with appropriate action or helpful comments. Chatbots perform the same magic in response to typed text queries.
- **Social media sentiment analysis:** Sentiment analysis can analyze language used in social media posts, responses, reviews, and more to extract attitudes and emotions in response to products, promotions, and events. companies can use this information in product designs, advertising campaigns, and more.
- **Text summarization:** Text summarization uses NLP techniques to digest huge volumes of digital text and create summaries and synopses for indexes, research databases, or busy readers who don't have time to read full text. The best text summarization applications use semantic reasoning and natural language generation (NLG) to add useful context and conclusions to summaries.

Applications of NLP:

- Machine Translation.
- Speech Recognition.
- Sentiment Analysis.
- Question Answering.
- Summarization of Text.
- Chatbot.
- Intelligent Systems.
- Text Classifications.
- Character Recognition.
- Spell Checking.
- Spam Detection.
- Autocomplete.
- Named Entity Recognition.
- Predictive Typing.

For a sentence it can be interpreted in many ways. Hence its is required for the NLP to understand the meaning of the context. Suppose consider the below statement.

Example 1:

I saw a man on a hill with a telescope.

- There is a man on the hill, and I watched him with my telescope.
- There is a man on the hill, and he has a telescope.
- I'm on a hill, and I saw a man using my telescope.
- I'm on a hill, and I saw a man who has a telescope.
- There is a man on a hill, and I saw him something with my telescope.

Can you help me with can?

- Here the first “can” word is used for question formation.
- The second “can” word at the end of the sentence is used to represent a container that holds food or liquid.
- In the sentence above, we can see that there are two “can” words, but both of them have different meanings. Here the first “can” word is used for question formation.
- The second “can” word at the end of the sentence is used to represent a container that holds food or liquid.
- Hence, from the examples above, we can see that language processing is not “deterministic” (the same language has the same interpretations), and something suitable to one person might not be suitable to another.
- Therefore, Natural Language Processing (NLP) has a non-deterministic approach. In other words, Natural Language Processing can be used to create a new intelligent system that can understand how humans understand and interpret language in different situations.

NLP-approaches rule based V/s statistical

- **Rule-based Natural Language Processing:** It uses common sense reasoning for processing tasks. For instance, the freezing temperature can lead to death, or hot coffee can burn people's skin, along with other common sense reasoning tasks. However, this process can take much time, and it requires manual effort.
- **Statistical Natural Language Processing:** It uses large amounts of data and tries to derive conclusions from it. Statistical NLP uses [machine learning](#) algorithms to train NLP models.
- After successful training on large amounts of data, the trained model will have positive outcomes.

Rule-based approaches are the oldest approaches to NLP. The rule-based or grammar-based approach implies that a human is involved in the process of stepwise system development and improvement.

A rule-based NLP system simply follows these rules to categorize the language it is analyzing. If the rule does not exist, the system will be unable to 'understand' the human language and thus will fail to categorize it. Regular expressions and context free grammars are some examples of rule-based approaches to NLP.

Advantages of Rule based approach:

- A rule-based system is good at capturing a specific language phenomenon: it will decode the linguistic relationships between words to interpret the sentence
 - It tends to focus on pattern-matching or parsing
 - Rule-based systems are low precision, high recall, meaning they can have high performance in specific use cases, but often suffer performance degradation when generalized
- Disadvantage of Rule-based approach:
- It requires skilled experts: it takes a linguist or a knowledge engineer to manually encode each rule in NLP.
 - Rules need to be manually crafted and enhanced all the time.
 - Moreover, the system can become so complex, that some rules can start contradicting each other.
 - Accuracy of the NLP system is dependent on the rules provided
 - They cannot easily scale to accommodate a seemingly endless stream of exceptions or the increasing volumes of text and voice data.

Statistical NLP: Statistical NLP aims to perform statistical inference for the field of NLP. It combines computer algorithms with machine learning and deep learning models to automatically extract, classify, and label elements of text and voice data and then assign a statistical likelihood to each possible meaning of those elements.

Today, deep learning models and learning techniques based on convolutional neural networks (CNNs) and recurrent neural networks (RNNs) enable NLP systems that 'learn' as they work and extract ever more accurate meaning from huge volumes of raw, unstructured, and unlabelled text and voice data sets. Advantages of Statistical NLP:

- The main advantage of statistical NLP is machine learning algorithm's "learnability", which is why no manual rule/grammar coding is needed, requiring high skills
- The corpus can be annotated using the low-skilled workforce.

- The data fed to such system will be huge and there are a lot of data points (e.g. keywords etc.), which makes it easy for the machine to learn statistical clues of the words for a given task.

- Machine learning approaches can significantly speed up the development of a capability of certain NLP systems, when good training data sets are available

Disadvantages of Statistical NLP

: • Lack of training data

- Poorly labelled, insufficient data

- New “preparation” of data is required each time as, once created and labelled, the corpus often cannot be reused on new data schemas

Which of the following tools are used in NLP to perform?

Natural Language Tool Kit or NLTK is one of the most popular NLP libraries in Python .

It supports a large amount of tasks and can be used to do anything from text pre-processing techniques like stopping word removal, tokenization, stemming, and lemmatization to building n-grams



- **SpaCy** - Data Extraction, Data Analysis, Sentiment Analysis, Text Summarization
- SpaCy is the next step of the NLTK evolution.
- NLTK is clumsy and slow when it comes to more complex business applications. At the same time, SpaCy provides users with a smoother, faster, and efficient experience.
- SpaCy, an open-source NLP library, is a perfect match for comparing customer profiles, product profiles, or text documents.
- SpaCy is good at **syntactic analysis**, which is handy for aspect-based **sentiment analysis and conversational user interface optimization**.
- SpaCy is also an excellent choice for **named-entity recognition**. You can use SpaCy for business insights and market research.

Text processing Task

- The term text processing refers to the automation of analysing electronic text. This allows **machine learning models** to get structured information about the text to use for analysis, manipulation of the text, or to generate new text.
- Text data derived from natural language is unstructured and noisy. Text pre-processing involves transforming text into a clean and consistent format that can then be fed into a model for further analysis and learning
- Text processing is one of the most common tasks used in machine learning applications such as language translation, sentiment analysis, spam filtering, and many others.
- Text pre-processing improves the performance of an NLP system.

What is the difference between text processing and natural language processing?

- Text processing refers to only the analysis, manipulation, and generation of text, while natural language processing refers to the ability of a computer to understand human language in a valuable way. Basically, natural language processing is the next step after text processing.
- The various pre-processing steps that are involved are:
 1. Lower Casing
 2. Tokenization
 3. Punctuation Mark Removal
 4. Stop Word Removal
 5. Stemming
 6. Lemmatization

Tokenization

- Tokenization is the process of breaking up the paragraph into smaller units such as sentences or words.
- Each unit is then considered as an individual token.
- The fundamental principle of Tokenization is to try to understand the meaning of the text by analysing the smaller units or tokens that constitute the paragraph.

Syntax:

`nltk.tokenize.sent_tokenize(text, language='english')`

returns a sentence-tokenized copy of text, using NLTK's recommended sentence tokenizer. (Currently Punkt Sentence Tokenizer for the specified language).

Parameters: Text (str) – text to split into sentences

Language(str) – the model name in the Punkt corpus. By default it will be English

```
#Import required libraries :  
import nltk  
from nltk import sent_tokenize  
from nltk import word_tokenize
```

```
#Tokenize the text by sentences :  
sentences = sent_tokenize(text)  
  
#How many sentences are there? :  
print (len(sentences))  
  
#Print the sentences :  
#print(sentences)  
sentences
```

Word tokenizing:

- Similarly, we can also tokenize the paragraph into words. The result is a list called 'words', containing each word of the paragraph. The length of the list gives us the total number of words present in our paragraph

Syntax:

`nltk.tokenize.word_tokenize(text, language='english', preserve_line=False)`

returns a tokenized copy of text, using NLTK's recommended word tokenizer (currently an improved TreebankWordTokenizer along with PunktSentenceTokenizer for the specified language).

Parameters text (str) – text to split into words

language (str) – the model name in the Punkt corpus

preserve_line (bool) – A flag to decide whether to sentence tokenize the text or not.

```
#Tokenize the text with words :  
words = word_tokenize(text)  
  
#How many words are there? :  
print (len(words))  
  
#Print words :  
print (words)
```

Find and visualize the frequency distribution of words

Frequency distribution table allows us to visualise the distribution of certain words in sentences. FreqDist is the function used to see the distribution of words in a sentence. Notice that the most used words are punctuation marks and stop words. We will have to remove such words to analyse the actual text.

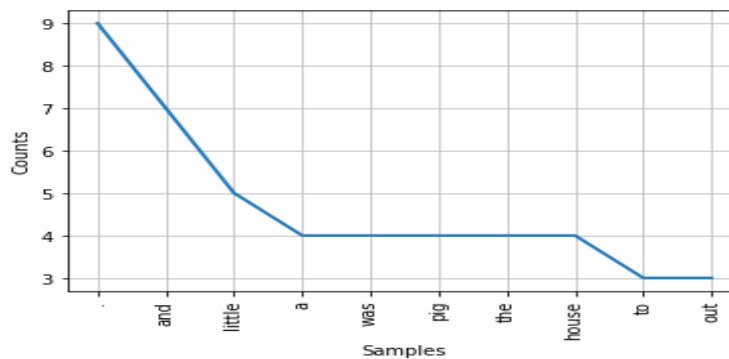
```
#Import required libraries :
from nltk.probability import FreqDist

#Find the frequency :
fdist = FreqDist(words)

#Print 10 most common words :
fdist.most_common(10)
```

```
#Plot the graph for fdist :
import matplotlib.pyplot as plt

fdist.plot(10)
```



Removing of Punctuation

- we are going to remove the punctuation marks as they are not very useful for us. We are using **isalpha()** method to separate the punctuation marks from the actual text.
- Also, we are going to make a new list called `words_no_punc`, which will store the words in lower case but exclude the punctuation marks.

```
#Empty list to store words:
words_no_punc = []

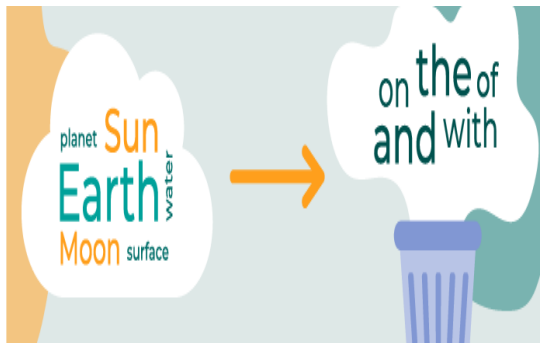
#Removing punctuation marks :
for w in words:
    if w.isalpha():
        words_no_punc.append(w.lower())

#Print the words without punctuation marks :
print (words_no_punc)

print ("\n")

#Length :
print (len(words_no_punc))
```

Stop words



Examples of a few stop words in English are “the,” “a”, “an”, “so”, “what”.

- Stop words are a set of commonly used words in any language. For example, in English, “the,” “is” and “and”, would easily qualify as stop words.
- In NLP and text mining applications, stop words are **used to eliminate unimportant words, allowing applications to focus on the important words instead.**
- **The words which are generally filtered out before processing a natural language** are called stop words.
- These are the most common words in any language (like articles, prepositions, pronouns, conjunctions, etc) and does not add much information to the text.

Removing Stop words

```
from nltk.corpus import stopwords

#List of stopwords
stopwords = stopwords.words("english")
print(stopwords)
```

```
#Empty list to store clean words :
clean_words = []

for w in words_no_punc:
    if w not in stopwords:
        clean_words.append(w)

print(clean_words)
print("\n")
print(len(clean_words))
```

Word Cloud:

- Word Cloud is a data visualization technique. In which words from a given text display on the main chart.
- In this technique, more frequent or essential words display in a larger and bolder font, while less frequent or essential words display in smaller or thinner fonts. It is a beneficial technique in NLP that gives us a glance at what text should be analysed.

Properties:

- `font_path`: It specifies the path for the fonts we want to use.
- `width`: It specifies the width of the canvas.
- `height`: It specifies the height of the canvas.
- `min_font_size`: It specifies the smallest font size to use.
- `max_font_size`: It specifies the largest font size to use.
- `font_step`: It specifies the step size for the font.
- `max_words`: It specifies the maximum number of words on the word cloud.
- `stopwords`: Our program will eliminate these words.
- `background_color`: It specifies the background color for canvas.
- `normalize_plurals`: It removes the trailing “s” from words.

Python code to display word cloud

```
#Library to form wordcloud :
from wordcloud import WordCloud

#Library to plot the wordcloud :
import matplotlib.pyplot as plt

#Generating the wordcloud :
wordcloud = WordCloud().generate(text)

#Plot the wordcloud :
plt.figure(figsize = (12, 12))
plt.imshow(wordcloud)

#To remove the axis value :
plt.axis("off")
plt.show()
```

the most frequent words display in larger fonts. The word cloud can be displayed in any shape or image.

Spell correction Normalization in NLP

Spell Correction

- There are many methods for spelling correction.
- This method takes a list of misspelled words and gives the suggestion of the correct word for each incorrect word.
- It tries to find a word in the list of correct spellings that has the shortest distance and the same initial letter as the misspelled word.
- It then returns the word which matches the given criteria.
- The methods can be differentiated on the basis of the distance measure they use to find the closest word. 'words' package from nltk is used as the dictionary of correct words.

Using Jaccard distance Method

- Jaccard distance, the opposite of the Jaccard coefficient, is used to measure the dissimilarity between two sample sets. We get Jaccard distance by subtracting the Jaccard coefficient from 1.
- We can also get it by dividing the difference between the sizes of the union and the intersection of two sets by the size of the union.
- We work with Q-grams (these are equivalent to N-grams) which are referred to as characters instead of tokens. Jaccard Distance is given by the following formula.
- **$D_j(A,B) = 1 - J(A,B) = (|A \cup B| - |A \cap B|) / |A \cup B|$**
- **Step 1:** First, we install and import the nltk suite and Jaccard distance metric that we discussed before.
- 'ngrams' are used to get a set of co-occurring words in a given window and ar# importing the nltk suite

```
In [1]: import nltk
```

```
In [3]: from nltk.metrics.distance import jaccard_distance  
        from nltk.util import ngrams
```

- Now, we download the 'words' resource (which contains the list of correct spellings of words) from the nltk downloader and import it through nltk.corpus and assign it to correct_words.

```
nltk.download('words')
from nltk.corpus import words
```

```
[nltk_data] Downloading package words to
[nltk_data] C:\Users\Shilpa\AppData\Roaming\nltk_data...
[nltk_data] Unzipping corpora\words.zip.
```

```
correct_words = words.words()
```

- Step 3:** We define the list of incorrect_words for which we need the correct spellings.
- Then we run a loop for each word in the incorrect words list in which we calculate the Jaccard distance of the incorrect word with each correct spelling word having the same initial letter in the form of bigrams of characters. We then sort them in ascending order so the shortest distance is on top and extract the word corresponding to it and print it.

```
correct_words = words.words()
```

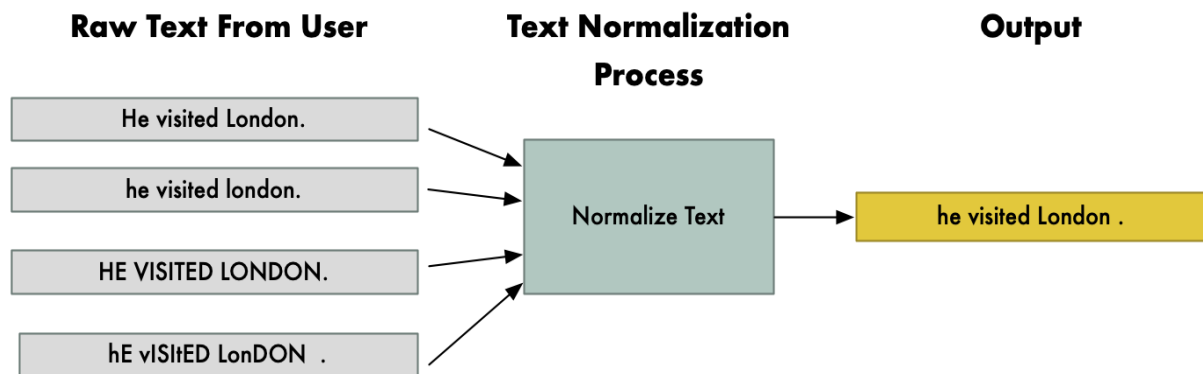
```
incorrect_words=['happy', 'azmaing', 'intelliengt']
```

```
for word in incorrect_words:
    temp = [(jaccard_distance(set(ngrams(word, 2)),
                                set(ngrams(w, 2))),w)
            for w in correct_words if w[0]==word[0]]
    print(sorted(temp, key = lambda val:val[0])[0][1])
```

```
happy
amazing
intelligent
```

Normalization in NLP

- Normalization is **the process of converting a token into its base form**. In the normalization process, the inflectional form of a word is removed so that the base form can be obtained.
- When we normalize text, **we attempt to reduce its randomness, bringing it closer to a predefined “standard.”** This helps us to reduce the amount of different information that the computer has to deal with, and therefore improves efficiency.



Why do we need text normalization?

- Normalization is helpful in reducing the number of unique tokens present in the text, removing the variations in a text and cleaning the text by removing redundant information.
- When we normalize text, we attempt to reduce its randomness, bringing it closer to a predefined “standard.”
- This helps us to reduce the amount of different information that the computer has to deal with, and therefore improves efficiency.

Two popular methods used for normalization are:

- **stemming**
- **lemmatization.**



Stemming

- Stemming is the **process of reducing the words to their word stem or root form**. The objective of stemming is to reduce related words to the same stem even if the stem is not a dictionary word.
- For example, connection, connected, connecting word reduce to a common word “connect.”
- Stemming is not a good process for normalization. since sometimes it can produce non-meaningful words which are not present in the dictionary.
- Stemming refers to a crude heuristic process that chops off the ends of words in the hope of achieving this goal correctly most of the time.

Implementation using Porter Stemmer The most common algorithm for stemming English is Porter’s algorithm. There are other stemmers also available such as Snowball Stemmer.

```

: from nltk.stem import PorterStemmer
  ps=PorterStemmer()

  w=["studies","studying","studied"]
  for i in w:
      print(ps.stem(i))

studi
studi
studi

```

```

: #Stemming Example :

#Import stemming library :
from nltk.stem import SnowballStemmer

snowball = SnowballStemmer("english")

#Word-list for stemming :
word_list = ["Study","Studying","Studies","Studied"]

for w in word_list:
    print(snowball.stem(w))

studi
studi
studi
studi

```

- **Over-stemming:** where a much larger part of a word is chopped off than what is required, which in turn leads to words being reduced to the same root word or stem incorrectly when they should have been reduced to more stem words.
- **For example, the words “university” and “universe” that get reduced to “univers”.**
- **Under-stemming:** occurs when two or more words could be wrongly reduced to more than one root word when they should be reduced to the same root word.
- For example, the words “data” and “datum” that get reduced to “dat” and “datu” respectively (instead of the same stem “dat”).

Lemmatization

- Unlike stemming, lemmatization reduces words to their base word, reducing the inflected words properly and ensuring that the root word belongs to the language.
- It is usually more sophisticated than stemming, since stemmers works on an individual word without knowledge of the context.
- Stemming does not consider the context of the word. That is why it generates results faster, but it is less accurate than lemmatization.

```
import nltk
nltk.download('wordnet')
nltk.download('omw-1.4')
from nltk import WordNetLemmatizer
```

```
from nltk import WordNetLemmatizer

lemma = WordNetLemmatizer()
word_list = ["studies", "leaves", "decreases", "plays"]

for w in word_list:
    print(lemma.lemmatize(w))
```

```
study
leaf
decrease
play
```

Parts of speech tagging in NLP

What is Part-of-speech (POS) tagging?

POS tagging is the task of assigning each word in a sentence the part of speech that it assumes in that sentence. The primary target of POS tagging is to identify the grammatical group of given word: whether it is a noun, pronoun, adjective, verb, adverbs, etc. based on the context.

Part of Speech	Tag
Noun	n
Verb	v
Adjective	a
Adverb	r

- Tagging is a kind of classification that may be defined as the automatic assignment of description to the tokens. Here the descriptor is called tag, which may represent one of the part-of-speech, semantic information and so on.
- It is a **process of converting a sentence to forms – list of words, list of tuples (where each tuple is having a form (word, tag))**. The tag in case of is a part-of-speech tag, and signifies whether the word is a noun, adjective, verb, and so on.

Why	not	tell	someone	?
adverb	adverb	verb	noun	punctuation mark, sentence closer

- Part-of-speech tags describe the characteristic structure of lexical terms within a sentence or text; therefore, we can use them for making assumptions about semantics. Other applications of POS tagging include:
- Named Entity Recognition
- Co-reference Resolution
- Speech Recognition

```
#PoS tagging :  
tag = nltk.pos_tag(["Studying","Study"])  
print (tag)  
  
[('Studying', 'VBG'), ('Study', 'NN')]
```

```
#PoS tagging example :

sentence = "A very beautiful young lady is walking on the beach"

#Tokenizing words :
tokenized_words = word_tokenize(sentence)

for words in tokenized_words:
    tagged_words = nltk.pos_tag(tokenized_words)

tagged_words

[('A', 'DT'),
 ('very', 'RB'),
 ('beautiful', 'JJ'),
 ('young', 'JJ'),
 ('lady', 'NN'),
 ('is', 'VBZ'),
 ('walking', 'VBG'),
 ('on', 'IN'),
 ('the', 'DT'),
 ('beach', 'NN')]
```

The following table provides the description of various tags produced using pos_tag() method of NLTK.

Tag	Description	Tag	Description
CC	Coordinating conjunction	PRP\$	Possessive pronoun
CD	Cardinal number	RB	Adverb
DT	Determiner	RBR	Adverb, comparative
EX	Existential <i>there</i>	RBS	Adverb, superlative
FW	Foreign word	RP	Particle
IN	Preposition or subordinating conjunction	SYM	Symbol
JJ	Adjective	TO	<i>to</i>
JJR	Adjective, comparative	UH	Interjection
JJS	Adjective, superlative	VB	Verb, base form
LS	List item marker	VBD	Verb, past tense
MD	Modal	VBG	Verb, gerund or present participle
NN	Noun, singular or mass	VBN	Verb, past participle
NNS	Noun, plural	VBP	Verb, non-3rd person singular present
NNP	Proper noun, singular	VBZ	Verb, 3rd person singular present
NNPS	Proper noun, plural	WDT	Wh-determiner
PDT	Predeterminer	WP	Wh-pronoun
POS	Possessive ending	WP\$	Possessive wh-pronoun
PRP	Personal pronoun	WRB	Wh-adverb

What is Chunking in NLP?

- **Chunking** in NLP is a process to take small pieces of information and group them into large units. The primary use of Chunking is making groups of “noun phrases.” It is used to add structure to the sentence by following POS tagging combined with regular expressions. The resulted group of words are called “chunks.” It is also called shallow parsing.
- In shallow parsing, there is maximum one level between roots and leaves while deep parsing comprises of more than one level. Shallow parsing is also called light parsing or chunking.
- **Rules for Chunking:**
- There are no pre-defined rules, but you can combine them according to need and requirement.
- For example, you need to tag Noun, verb (past tense), adjective, and coordinating junction from the sentence. You can use the rule as below
- chunk: {<NN.?>*<VBD.?>*<JJ.?>*<CC>?}

Use Case of Chunking

- Chunking is used for entity detection. An entity is that part of the sentence by which machine get the value for any intention.
- Example:
- Temperature of New York.
- Here Temperature is the intention and New York is an entity.

Use Case of Chunking

Chunking is used for entity detection. An entity is that part of the sentence by which machine get the value for any intention.

Example:

Temperature of New York.

Here Temperature is the intention and New York is an entity.

```
import nltk
text = "learn php from guru99"
tokens = nltk.word_tokenize(text)
print(tokens)
tag = nltk.pos_tag(tokens)
print(tag)
grammar = "NP: {<DT>?<JJ>*<NN>}"
cp = nltk.RegexpParser(grammar)
result = cp.parse(tag)
print(result)
result.draw()    # It will draw the pattern graphically which can be seen in Noun Phrase
                  chunking
```

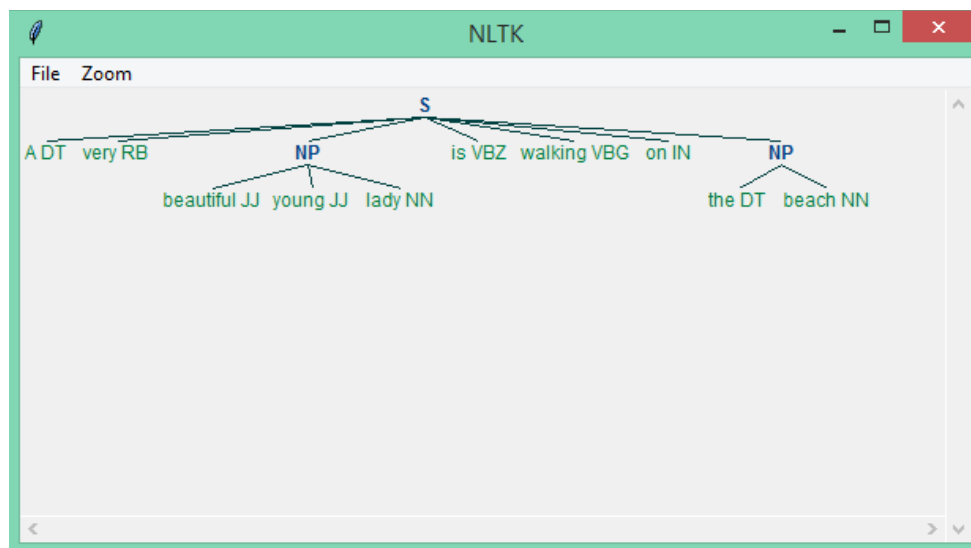
```
#Extracting Noun Phrase from text :
# ? - optional character
# * - 0 or more repetitions
grammar = "NP : {<DT>?<JJ>*<NN>} "

#Creating a parser :
parser = nltk.RegexpParser(grammar)

#Parsing text :
output = parser.parse(tagged_words)
print (output)

#To visualize :
output.draw()
```

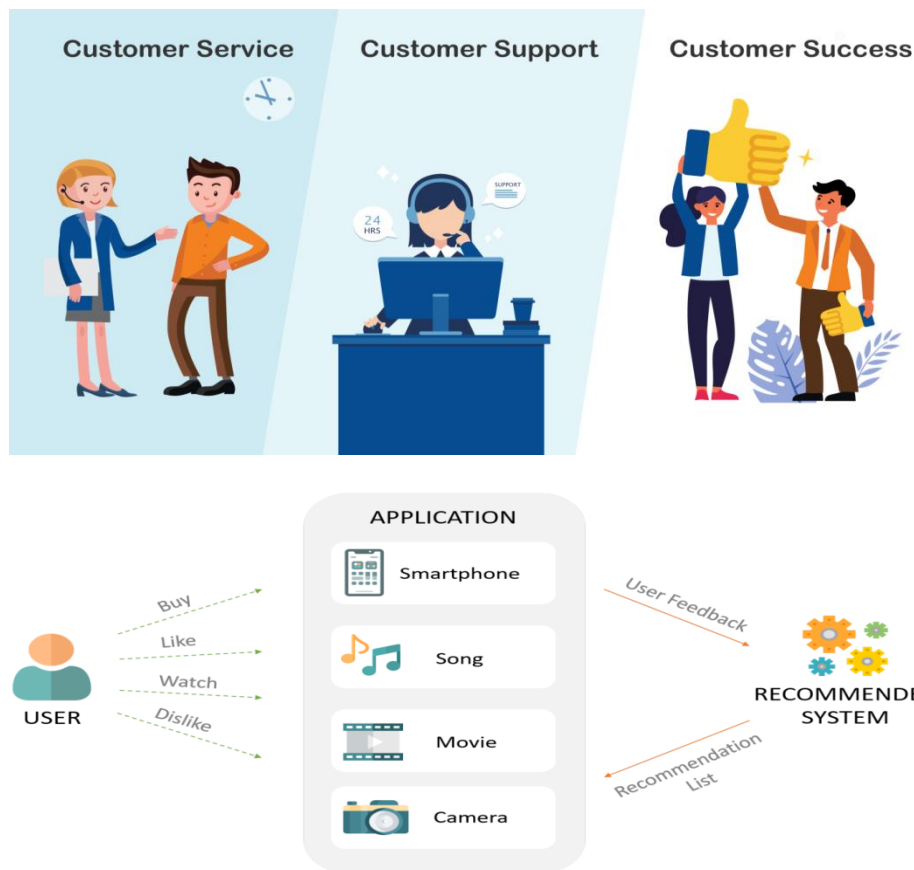
(S
 A/DT
 very/RB
 (NP beautiful/JJ young/JJ lady/NN)
 is/VBZ
 walking/VBG
 on/IN
 (NP the/DT beach/NN))



Named Entity Recognition

- The named entity recognition (NER) is one of the most popular data pre-processing task. **It involves the identification of key information in the text and classification into a set of predefined categories.**
- An entity is basically the thing that is consistently talked about or refer to in the text. NER is the form of NLP
- NER can also **help you to better understand the structure of your text data and to find relationships between entities.**
- In addition, NER can be used to improve the accuracy of other NLP tasks, such as part-of-speech tagging and dependency parsing.

Named entities are proper nouns that refer to specific entities that can be a person, organization, location, date, etc. Consider this example – “*Mount Everest is the tallest mountain*”. Here *Mount Everest* is a named entity of type location as it refers to a specific entity.



Named entities are persons, locations, organizations, time expressions, etc. POS tagger does not look for the relation between the words in the document whereas NER looks for the relationship between words. The output of POS tagging is used as an input for NER

The Kashmir Files is a **2022** **CARDINAL** Indian Hindi-language drama film, written and directed by **Vivek Agnihotri** **PERSON**. Produced by **Zee Studios** **ORG**, the film is based on the exodus of **Kashmiri** **NORP** Pandits during **the Kashmir Insurgency** **GPE**, which it portrays as a genocide. It stars **Anupam Kher** **PERSON**, **Darshan Kumar** **PERSON**, **Pallavi Joshi** **PERSON** and **Mithun Chakraborty** **PERSON**.

- Some other examples of named entities are listed below in the table.

	Named Entity:	Examples
• 1	ORGANIZATION	SEI, BCCI, Australia Cricket Board
• 2	PERSON	Barack Obama, Narendra Modi, Kohli
• 3	MONEY	7 million dollars, INR 7 Crore
• 4	GPE	India, Australia, South East Asia
• 5	LOCATION	Mount Everest, River Nile
• 6	DATE	8th June 1998, 7 April
• 7	TIME	8:45 A.M., two-fifty am

Named Entity Recognition is useful in –

- The field of academics by easy and faster extraction of information for the students and researchers from the searching data.
- In Question Answer system to provide answers from the data by the machine and hence minimizing human efforts.
- In content classification by identifying the theme and subject of the contents and makes the process faster and easy, suggesting the best content of interest.
- Helps in customer service by categorizing the user complaint, request, and question in respective fields and filtering by priority keywords.
- Helps to categories the books and articles in the e-library on different subjects and thus making it organized.
- NLTK provides a function `nltk.ne_chunk()` that is already a pre-trained classifier to recognize named entity using POS tag as input.

```
import nltk
from nltk import word_tokenize,pos_tag

text = "NASA awarded Elon Musk's SpaceX a $2.9 billion contract to build the lunar
tokens = word_tokenize(text)
tag=pos_tag(tokens)
print(tag)

ne_tree = nltk.ne_chunk(tag)
print(ne_tree)
```

- In the output, we can see that the classifier has added category labels such as PERSON, ORGANIZATION, and GPE (geographical physical location) where ever it founded named entity.

```
#Sentence for NER :
sentence = "Mr. Smith made a deal on a beach of Switzerland near WHO."

#Tokenizing words :
tokenized_words = word_tokenize(sentence)

#PoS tagging :
for w in tokenized_words:
    tagged_words = nltk.pos_tag(t_w)

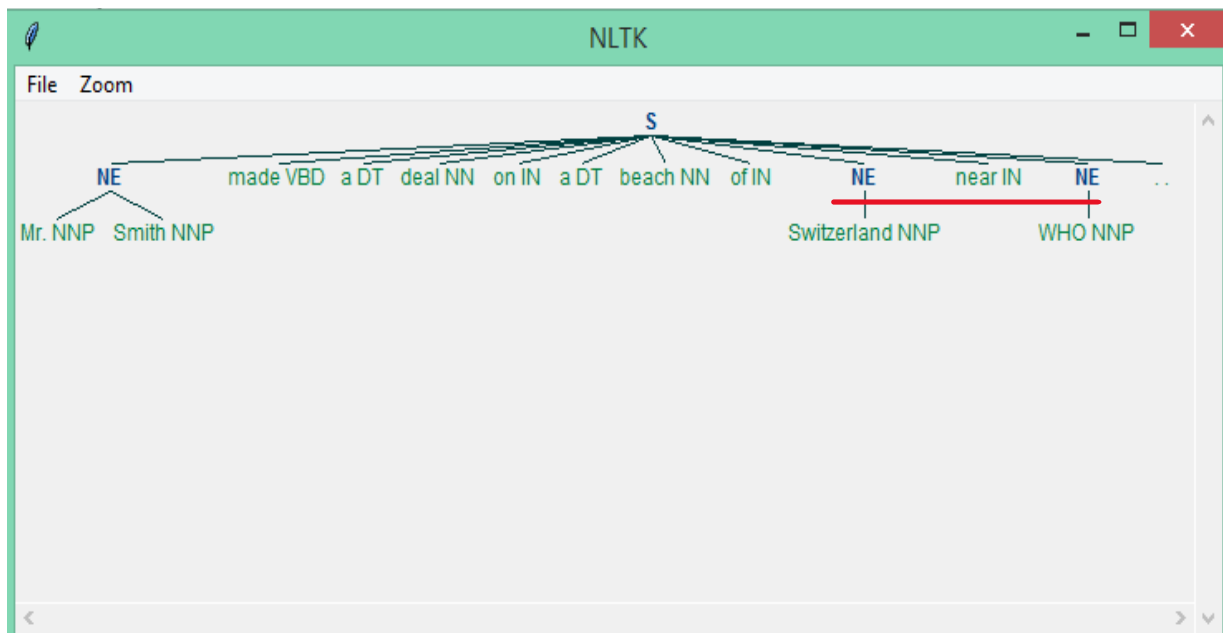
#print (tagged_words)

#Named Entity Recognition :
N_E_R = nltk.ne_chunk(tagged_words, binary=True)

print(N_E_R)

#To visualize :
N_E_R.draw()
```

```
(S
 (NE Mr./NNP Smith/NNP)
 made/VBD
 a/DT
 deal/NN
 on/IN
 a/DT
 beach/NN
 of/IN
 (NE Switzerland/NNP)
 near/IN
 (NE WHO/NNP)
 ./.)
```



```

#Sentence for NER :
sentence = "Mr. Smith made a deal on a beach of Switzerland near WHO."

#Tokenizing words :
tokenized_words = word_tokenize(sentence)

#PoS tagging :
for w in tokenized_words:
    tagged_words = nltk.pos_tag(t_w)

#print (tagged_words)

#Named Entity Recognition :
N_E_R = nltk.ne_chunk(tagged_words, binary=False)
print(N_E_R)

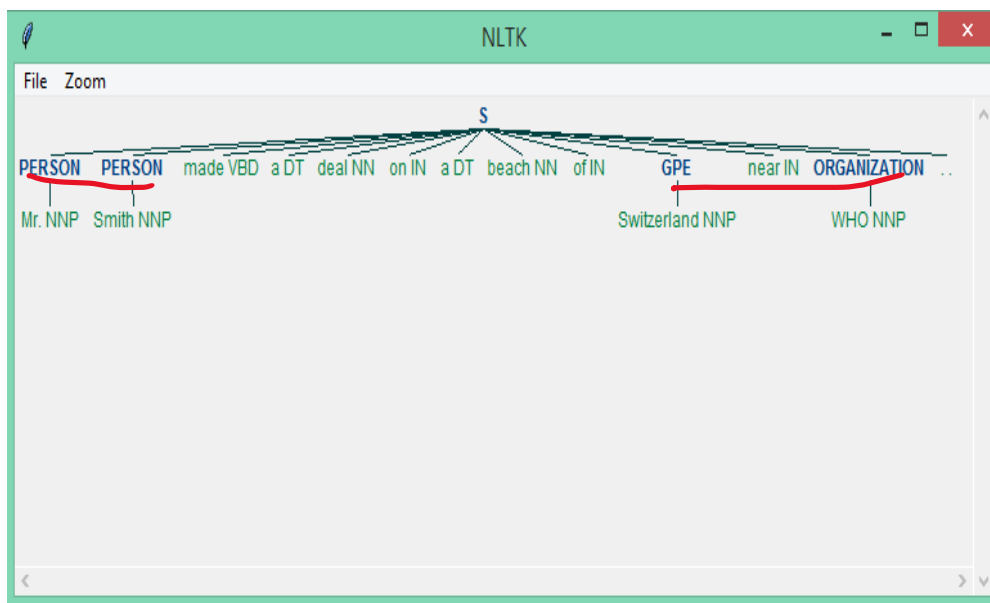
#To visualize :
N_E_R.draw()

```

```

(S
 (PERSON Mr./NNP)
 (PERSON Smith/NNP)
 made/VBD
 a/DT
 deal/NN
 on/IN
 a/DT
 beach/NN
 of/IN
 (GPE Switzerland/NNP)
 near/IN
 (ORGANIZATION WHO/NNP)
 ./.)

```



VECTORIZ ER

Vectorization or word embedding is jargon for a classic approach of converting input data from its raw format (i.e. text) into vectors of real numbers which is the format that Machine Learning models support.

- After text cleaning and normalization, the processed text is converted feature vectors so that we can feed it to machine learning applications.
- In Machine Learning, vectorization is a step, in feature extraction. The idea is to get some distinct features out of the text for the model to train on, by converting text to numerical vectors.

Why do we need Word Embeddings?

- As we know that many Machine Learning algorithms and almost all Deep Learning Architectures are not capable of processing strings or plain text in their raw form.
- They require numerical numbers as inputs to perform any sort of task, such as classification, regression, clustering, etc.
- Also, from the huge amount of data that is present in the text format, it is imperative to extract some knowledge out of it and build any useful applications.
- we can say that to build any model in machine learning or deep learning, the final level data has to be in numerical form because models don't understand text or image data directly as humans do.

Word Embedding Techniques

- One-hot Encoding (OHE)
- Count Vectorizer
- Bag-of-Words (BOW)
- N-grams
- Term Frequency-Inverse Document Frequency (TF-IDF)

One-Hot Encoding

- In this technique, we represent each unique word in vocabulary by setting a unique token with value 1 and rest 0 at other positions in the vector.
- In simple words, a vector representation of a one-hot encoded vector represents in the form of 1, and 0 where 1 stand for the position where the word exists and 0 everywhere else.

Consider the example:

Sentences=' I am teaching NLP in Python'

Tokens=['I','am','teaching','NLP','in','Python']

Vector of NL=[0,0,0,1,0,0]

Vector for python=[0,0,0,0,0,1]

Disadvantages of One-hot Encoding

- 1. One of the disadvantages of One-hot encoding is that the Size of the vector is equal to the count of unique words in the vocabulary.
- 2. One-hot encoding does not capture the relationships between different words. Therefore, it does not convey information about the context.

Count Vectorizer

- It is one of the simplest ways of doing text vectorization.
- It creates a document term matrix, which is a set of dummy variables that indicates if a particular word appears in the document.
- Count vectorizer will fit and learn the word vocabulary and try to create a document term matrix in which the individual cells denote the frequency of that word in a particular document, which is also known as term frequency, and the columns are dedicated to each word in the corpus.

Let's consider the following example:

Sentence 1: He is a smart boy. She is also smart.

Sentence-2: Chirag is a smart person.

The dictionary created contains the list of unique tokens(words) present in the corpus
Unique Words: ['He', 'She', 'smart', 'boy', 'Chirag', 'person']

Here, $s=2$, $N=6$ So, the count matrix M of size 2×6 will be represented as –

	He	She	Smart	Boy	Chirag	person
s1	1	1	2	1	0	0
s2	0	0	1	0	1	1

Now, a column can also be understood as a word vector for the corresponding word in the matrix M . For Example, for the above matrix formed, let's see the word vectors generated.

Vector for 'smart' is [2,1], Vector for 'Chirag' is [0, 1], and so on.

```
from sklearn.feature_extraction.text import CountVectorizer
```

```
sents = ['he is a smart boy . she is also smart',
         'srinandan is a smart person']
```

```
cv = CountVectorizer()
```

```
X = cv.fit_transform(sents)
X = X.toarray()
```

```
vocabulary=sorted(cv.vocabulary_.keys())
```

```
print(vocabulary)
print(X)
```

```
['also', 'boy', 'he', 'is', 'nandan', 'person', 'she', 'smart', 'sri']
[[1 1 1 2 0 0 1 2 0]
 [0 0 0 1 1 1 0 1 1]]
```

N-grams

- N-grams are continuous sequences of words or symbols or tokens in a document. In technical terms, they can be defined as the neighbouring sequences of items in a document.
- They come into picture when we deal with text data in NLP (Natural Language Processing) tasks.
- An N-gram means **a sequence of N words**. So for example, “Medium blog” is a 2-gram (a bigram), “A Medium blog post” is a 4-gram, and “Write on Medium” is a 3-gram (trigram).
Well, that was not very interesting or exciting.

“I reside in Bengaluru”

SL.No.	Type of n-gram	Generated n-grams
1	Unigram	["I","reside","in", "Bengaluru"]
2	Bigram	["I reside","reside in","in Bengaluru"]
3	Trigram	["I reside in", "reside in Bengaluru"]

- Similar to the count vectorization technique, in the N-Gram method, a document term matrix is generated and each cell represents the count. The difference in the N-grams method is that the count represents the combination of adjacent words of length n in the title.

```
from sklearn.feature_extraction.text import CountVectorizer
```

```
sents = ['he is a smart boy . she is also smart',  
        'srinandan is a smart person']
```

```
cv = CountVectorizer(ngram_range=(2,2))
```

```
X = cv.fit_transform(sents)  
X = X.toarray()
```

```
vocabulary=sorted(cv.vocabulary_.keys())
```

```
print(vocabulary)  
print(X)
```

```
['also smart', 'boy she', 'he is', 'is also', 'is smart', 'she is', 'smart boy', 'smart person', 'srinandan is']  
[[1 1 1 1 1 1 1 0 0]  
 [0 0 0 0 1 0 0 1 1]]
```

TF-IDF

- TF-IDF stands for Term Frequency—Inverse Document Frequency, which is a scoring measure generally used in information retrieval (IR) and summarization. The TF-IDF score shows how important or relevant a term is in a given document.

The intuition behind TF and IDF:

- If a particular word appears multiple times in a document, then it might have higher importance than the other words that appear fewer times (TF). At the same time, if a particular word appears many times in a document, but it is also present many times in some other documents, then maybe that word is frequent, so we cannot assign much importance to it. (IDF).

$$TF = \frac{\text{Frequency of the word in the sentence}}{\text{Total number of words in the sentence}}$$

- IDF stands for Inverse Document Frequency, but before we go into IDF, we must make sense of DF – Document Frequency.

$$DF = \frac{\text{Documents containing word } W}{\text{Total number of documents}}$$

- DF tells us about the proportion of documents that contain a certain word.
- It is the reciprocal of the Document Frequency, and the final IDF score comes out of the following formula.

$$IDF = \log\left(\frac{\text{Total number of documents}}{\text{Documents containing word } W}\right)$$

Just as we discussed above, the intuition behind it is that the more common a word is across all documents, the lesser its importance is for the current document.

A logarithm is taken to dampen the effect of IDF in the final calculation.

The final TF-IDF score comes out to be:

$$TF - IDF = TF * IDF$$

This is how TF-IDF manages to incorporate the significance of a word. The higher the score, the more important that word is.

Program to demonstrate TF-IDF

```
from sklearn.feature_extraction.text import TfidfVectorizer
sents = ['coronavirus is a highly infectious disease',
         'coronavirus affects older people the most',
         'older people are at high risk due to this disease']
```

```
tfidf = TfidfVectorizer()
transformed = tfidf.fit_transform(sents)

import pandas as pd
df = pd.DataFrame(transformed[0].T.todense(),
                  index=tfidf.get_feature_names_out(),
                  columns=["TF-IDF"])
df = df.sort_values('TF-IDF', ascending=False)
df
```

Output:

	TF-IDF
infectious	0.490479
highly	0.490479
is	0.490479
coronavirus	0.373022
disease	0.373022
older	0.000000
this	0.000000
the	0.000000
risk	0.000000
people	0.000000
affects	0.000000
most	0.000000
are	0.000000
high	0.000000
due	0.000000
at	0.000000
to	0.000000